

Memory-Efficient Versioned File Indexer in C++

1. Introduction

This report describes the design and implementation of a memory-efficient, versioned word-level file indexer written in C++. The program processes large text files incrementally using a fixed-size buffer, builds a word-frequency index, and supports three analytical query types without ever loading the entire file into memory.

2. System Design

2.1 Class Architecture

Class	Role
BufferFileReader	Reads the file in fixed-size binary chunks. Handles words split at buffer boundaries using a leftover string.
Tokenizer	Splits a text chunk into alphanumeric tokens. Non-alphanumeric characters act as delimiters.
VersionedIndexer	Maintains an <code>unordered_map<string, long long></code> frequency map for one named version.
QueryProcessor (abstract)	Abstract base class with a pure virtual <code>execute()</code> method.
WordQuery / TopKQuery / DiffQuery	Concrete subclasses implementing word-count, top-K, and cross-version diff queries.

2.2 Inheritance & Polymorphism

QueryProcessor is an abstract base class with a pure virtual `execute()` method. WordQuery, TopKQuery, and DiffQuery each inherit from it and override `execute()`. In `main()`, all three are accessed through a QueryProcessor* pointer, enabling runtime polymorphism via dynamic dispatch.

2.3 Exception Handling

All runtime errors are handled with `try/catch/throw`:

- Buffer size outside [256, 1024] KB — throws `std::invalid_argument`
- File not found or unreadable — throws `std::runtime_error`
- Missing required arguments — throws `std::invalid_argument`

3. Buffer & Memory Strategy

A single fixed-size `char[]` buffer is allocated once and reused for every read. To handle words split across buffer boundaries, a leftover string carries any partial word from one chunk into the next. Memory usage grows only with the number of unique words, not with file size.

4. Supported Query Types

4.1 Word Count (--query word)

Returns the frequency of a given word (case-insensitive) in a version.

```
./analyzer --file dataset_v1.txt --version v1 --buffer 512 --query word --word error
```

4.2 Top-K (--query top)

Returns the K most frequent words, sorted descending. Ties broken alphabetically.

```
./analyzer --file dataset_v1.txt --version v1 --buffer 512 --query top --top 10
```

4.3 Difference (--query diff)

Reports $\text{count}(v2) - \text{count}(v1)$ for a word across two separately indexed files.

```
./analyzer --file1 dataset_v1.txt --version1 v1 \  
--file2 dataset_v2.txt --version2 v2 \  
--buffer 512 --query diff --word error
```

5. Input / Output

5.1 Command-Line Arguments

Argument	Description
--file <path>	Input file (single-version queries)
--file1 / --file2 <path>	Input files for diff query
--version <n>	Version identifier
--version1 / --version2 <n>	Version identifiers for diff query
--buffer <kb>	Buffer size in KB (256-1024)
--query <type>	Query type: word top diff
--word <token>	Word to look up
--top <k>	Number of top results

5.2 Output Format

```
Version: v1  
Query result: error -> 142  
Buffer size (KB) : 512  
Execution time (s): 0.384
```

6. Conclusion

The system meets all core assignment requirements: buffered incremental file reading, case-insensitive word indexing, version management, and three query types. The design demonstrates inheritance, runtime polymorphism, and exception handling in idiomatic C++, with memory usage independent of file size.